QuanSkill · AI Engineering · Week 3 — Chapter 5 Baseline Homework: Written Response

Name: Nguyen Tien Viet Hung **Cohort:** K10 **Date:** 02/10 **Track:** Customer Support Assistant

Run record. Notebook `QuanSkill_Chapter5_ML_Baselines_Executed_Hung.ipynb`, restarted and run top to bottom with `USE_UPLOAD = False` (built-in **synthetic teaching data**). Seeds: GroupShuffleSplit 42 / 43, all models `random_state=42`. Environment: Python 3.13, scikit-learn 1.9.1, numpy 2.5.3, pandas 3.0.5. **Edits:** no original cell changed; a clearly labelled section 11 (student analysis, read-only) was appended. The controlled experiment (2D) is a separate copy. All metrics below are from this teaching run, not from the book. Printed page p = PDF page p + 26.

Part 1 — Read and calculate

1A Baseline ladder. y = 1 when the candidate passage contains evidence that is useful for answering the question (sharing topic words is not enough; eq. 5.20, p. 420). **No-skill:** majority class (always "irrelevant"). **Lexical heuristic:** token coverage $|Q \cap P|/|Q|$ with a threshold (eq. 5.3, p. 408); TF–IDF cosine is the stronger lexical reference. **Learned classical model:** logistic regression on engineered pair features. To justify itself, the learned model must beat both simpler methods on the **same validation questions and metric**. That is evidence that learned weighting adds signal beyond doing nothing and beyond keyword matching (pp. 407, 422). **Keyword match but irrelevant:** for "Why are attention scores divided by $\sqrt{d_k}$?", the passage "Multi-head attention runs several attention functions in parallel…" shares *attention* but gives no reason for scaling (a hard negative, p. 499).

1B Confusion matrix (TP = 24, FP = 9, TN = 57, FN = 10; N = 100).

Metric	Substitution	Result
Accuracy	(TP+TN)/N = (24+57)/100	81/100 = **0.810**
Precision	TP/(TP+FP) = 24/(24+9)	24/33 = **0.727**
Recall	TP/(TP+FN) = 24/(24+10)	24/34 = **0.706**
Specificity	TN/(TN+FP) = 57/(57+9)	57/66 = **0.864**
F1	2TP/(2TP+FP+FN) = 48/(48+9+10)	48/67 = **0.716** (= 2·0.727·0.706/(0.727+0.706))

If missing relevant evidence costs more, I would prioritise **reducing false negatives (raise recall)**, for example by lowering τ on validation or minimising C(τ) = c_FN·FN + c_FP·FP (eq. 5.52, p. 464). **Trade-off:** more false positives, which means lower precision and more irrelevant context for reviewers or the generator to read.

1C Threshold reasoning. Scores [0.82, 0.68, 0.54, 0.43, 0.21] → τ = 0.50: **[1, 1, 1, 0, 0]**; τ = 0.70: **[1, 0, 0, 0, 0]**. Raising τ predicts fewer positives. Recall can only stay the same or fall, because positives below the new cutoff become FN. Precision *often* rises because borderline items are removed, but it can also fall: if the removed items (0.68, 0.54) were true positives, precision drops. The scores need only rank items for this to hold; they do not have to be calibrated. **To compute the metrics exactly I need the true label of each of the five items** (and, for a full set, the labels of all scored items).

Part 2 — Run and challenge the baseline

2A Data and split audit. 192 pairs, **48 questions**, 48 document IDs; prevalence **0.250** (48 relevant / 144 irrelevant). Grouped split: **train 33 questions (132 rows), validation 7 (28 rows), test 8 (32 rows)**, each with prevalence 0.250. Assertion result: **all three question_id sets are pairwise disjoint (assertions passed, True).** Splitting rows at random would put three of a question's four candidate passages in training and one in test. The model would then have already seen that exact question wording paired with its answer and negatives, so the test would measure recall of memorised question–passage patterns, not behaviour on a *new question*. Because pairs from one question are correlated, the effective number of independent test cases is also much smaller than the row count, which makes the estimate look more precise and more optimistic than it is. Another real boundary is the **source document**: in my run **12 document IDs appear in both train and test**, so a document-held-out split is needed to claim generalisation to new documents. Time also matters (questions about a policy version written after the training snapshot).

2B Validation ladder (7 questions, 28 pairs; threshold chosen on validation F1).

Method	Val. F1	Recall	Precision	Avg. precision	Threshold
Majority	0.000	0.000	0.000	0.250	0.50 (fixed)
Token coverage	0.667	0.714	0.625	0.589	0.13
TF–IDF similarity	0.588	0.714	0.500	0.390	0.01
Engineered logistic	0.667	0.714	0.625	0.679	0.40
Sparse TF–IDF logistic	0.483	1.000	0.318	0.246	0.44
Shallow tree	0.714	0.714	0.714	0.554	0.21
Random forest (SELECTED)	**0.769**	0.714	0.833	0.766	**0.56**

Selection rule: the highest validation F1 wins; ties go to the method listed earlier (simpler) on the baseline ladder. Here token coverage and engineered logistic tie at 0.667, so coverage would rank above it. The random forest wins outright (0.769) with the best AP (0.766) and was **frozen before the test cell**. This cannot establish real-world superiority: the validation set has only **7 positives**, so one flipped pair moves F1 by about 0.1. The threshold was tuned on the same 28 pairs, which makes the winning F1 optimistic. The corpus is synthetic, with passage templates repeated across questions. Recall is identical (0.714 = 5/7) for five methods, so the ranking rests on 1–4 false positives.

2C Held-out test (once, frozen model and τ = 0.56). Confusion matrix: **TP 4, FP 0, TN 24, FN 4**. Test **F1 = 0.667**, **recall = 0.500**, **average precision = 0.804** (precision 1.000, specificity 1.000, ROC-AUC 0.857, Brier 0.103). No model or threshold was changed afterwards. `week03_relevance_baseline.joblib` was saved and reloaded. The reload demo scored the example pair 0.0392 → not relevant, and re-scoring all 32 test pairs after reload gave **identical scores and predictions**. To reproduce one prediction I need: (1) the **fitted train-only TF–IDF vectorizer** (vocabulary, IDF, ngram (1, 2), min_df = 2); (2) the **feature code version** `chapter5_v1`, i.e. the token regex and six pair features, which live in the notebook code and *not* inside the joblib file; (3) the **fitted random-forest estimator** and the library versions (scikit-learn 1.9.1); (4) the **threshold 0.56** plus the input contract (raw question and passage strings, positive class = 1). I trust this joblib file only because my own run produced it.

2D One controlled experiment. Changed **only** the regularisation of the sparse TF–IDF logistic model: C = 1.0 → **C = 10.0**. Same seeds, split and everything else, re-run from the split onward. **Hypothesis:** if that model's near-chance AP was caused by over-regularising 1,118 sparse features on 132 rows, weaker regularisation would raise AP. If the cause is the representation (concatenated pair text encodes topic words, not whether *this* passage answers *this* question), AP would stay near prevalence. **Result (validation):** F1 0.483 → 0.500, AP 0.246 → 0.260, ROC-AUC 0.469 → 0.510, threshold 0.44 → 0.23. The model is still about at chance (prevalence 0.25), which supports the representation explanation. The selected model did not change (random forest, 0.769). The test numbers printed by run 2 are **exploratory only**; run 1's test is the only formal result.

Part 3 — Investigate errors

The frozen model made **0 false positives and 4 false negatives** on test. For the FP rows I therefore inspected the two **highest-scoring true negatives** (near misses, not errors), plus two of the four FNs.

Case and source	Observed evidence	Likely cause	Check or remedy
FP 1 → near miss (TN) q06_1 · demo_doc_06_1 · p. 2 · score 0.352	Q: why are missed relevant passages costly? P: "Precision asks what fraction of selected passages are truly relevant." Coverage 0.50, cosine 0.27; correctly below τ.	Hard negative: same topic (evaluation metrics), shared words *relevant/passages*, no answer.	Add more hard negatives to training; watch this band (0.30–0.36) if τ is lowered.
FP 2 → near miss (TN) q06_3 · demo_doc_06_1 · p. 2 · score 0.314	Q: what fraction of relevant evidence is found? P: same precision sentence. Coverage 0.57, cosine 0.41.	Keyword overlap without the right concept (precision vs recall).	Hard-negative mining; a feature that cannot separate these needs a better representation.
FN 1 q09_1 · demo_doc_09_0 · p. 1 · score 0.304	Q: what does an eighty percent probability mean? P (labelled 1): "Brier score averages squared differences…". Zero shared tokens. The passage that really answers it ("about eighty percent of comparable cases…") is paired with other variants.	**Label quality:** the generator assigns `positives[variant % 2]` by index, not by meaning.	Re-annotate topic 09 pairs with a written guide and 2 reviewers before scoring models.
FN 2 q01_2 · demo_doc_01_0 · p. 1 · score 0.407	Q: how is early sequence information lost? P: "Repeated multiplication by derivatives smaller than one… early time steps shrink…". Only *early* shared (coverage 0.17, cosine 0).	**Lexical paraphrase** (the same case as p. 499), close to the threshold.	Semantic representation in Week 4; threshold sensitivity checked on validation only.

3B Separating causes. (1) **Data/label quality — q09_1 and q09_3** (both FN, scores 0.304 and 0.089, the same Brier passage). For q09_3 ("Why is ranking distinct from calibration?") the Brier sentence does not explain the distinction. The labels follow the generator's index rule, so no model should be blamed for missing them. A clue: the *same* Brier passage scores exactly 0.304 both as a "positive" (q09_1) and as a negative for q06_3. With zero lexical overlap, the six features reduce to token counts, so the score cannot depend on meaning. (2) **Lexical representation — q06_1** ("missed relevant passages costly" vs "a false negative removes a useful passage…", score 0.089, coverage 0, cosine 0). The answer is a paraphrase, and at 0.089 it is a **ranking** failure, not a threshold failure (p. 501). (3) **Threshold choice** explains at most q01_2 (0.407): a diagnostic sweep shows τ = 0.40 would recover it with 0 FP. That is a reason to re-tune on more validation data, **not** to change τ after seeing test. **Targeted experiment:** freeze the current test set. Relabel topic 09 (and audit the other topics) using train/validation questions only. Then build a validation slice of positives with zero lexical overlap and compare the current forest with an embedding-similarity feature on it. If the embedding wins only on the paraphrase slice, representation is the cause; if both models fail on relabelled pairs, labels are. A claim like "use a larger model" is supported only by the paraphrase pattern (2 of 4 FNs), not by the label errors.

3C Question-level reliability. Pair recall = **0.500**; question hit rate = **0.500** (4 of 8 test questions have ≥ 1 relevant passage found). They are equal here only because every test question has exactly one positive. **Realistic contrast:** a question with 8 relevant chunks spread across a policy manual where the system finds 2 has pair recall 0.25, but the hit is 1, and an agent or researcher who needs *one* citable passage is served. That is why question-level success matters for retrieval (eq. 5.59, p. 502). **Limitation:** a hit counts one weak passage as success and ignores rank and the irrelevant passages shown next to it. Pair recall ignores which questions fail completely. Neither tells a user whether the evidence shown is *sufficient* to trust the answer.

Part 4 — Decide what to build next

4A Track adaptation — Customer Support Assistant (book Track C, p. 509).

Element	Design for my track
Target and time point	Multiclass **ticket intent** (8 intents from W2, e.g. refund_status, delivery_time) predicted from the **first customer message at ticket arrival**, before any agent reply; it routes the agent's question to the right knowledge-base category for retrieval.
No-skill / heuristic	Majority intent (train) · keyword rules per intent (e.g. “refund”, “track”, “password”).
Classical model	TF–IDF (fit on train only) + multinomial logistic regression.
Leakage-safe split	W2 **temporal split grouped by ticket** (train < 1 Aug, validation = Aug, test = Sep); agent replies, resolution and CSAT excluded (post-outcome).
Primary metric	**Macro-F1**, so rare intents (account_access, other) count equally, plus per-class recall and the confusion matrix (eq. 5.66).
Error to inspect	refund_status vs payment_issue confusion (“charged twice, want my money back”): a taxonomy or label problem rather than a model problem (p. 510).

Difference from the notebook: one text input instead of a (question, passage) pair, eight classes instead of binary relevance, a time and ticket boundary instead of a question group, and macro-F1 instead of binary F1/AP. Caveat: my W2 synthetic tickets have only 26 question wordings, so any score on them would be inflated.

4B Acceptance review (pp. 516–518, for my actual run).

Item	Status	Evidence or limitation
Label definition	**Partial**	y defined precisely (eq. 5.20), but synthetic labels are assigned by index; the audit found mislabelled topic-09 pairs.
Protected split	**Met** (question level)	Disjoint question_id assertions passed; but 12 documents are shared between train and test, so this is not a document holdout.
No-skill comparison	**Met**	Majority: F1 0.000, AP 0.250 (= prevalence) on the same validation set.
Train-only fitted preprocessing	**Met**	Vectorizer fitted on train text; scaler and TF–IDF inside pipelines; GroupKFold refits per fold.
Validation-selected threshold	**Met**	$\tau = 0.56$ and the model frozen on validation before the single test evaluation.
Saved inference artifact	**Partial**	joblib saved, reloaded and re-scored identically, but the feature functions and library version must be shipped with it (p. 516, Failure 12).

The synthetic demo passes most **procedural** checks, but it is **weak evidence for deployment**: 8 test questions, repeated templates, and labels that are known to be imperfect.

4C Business decision. Value = 75 h × \$20/h = **\$1,500/month**; extra cost = \$1,200/month; **net = \$1,500 − \$1,200 = +\$300/month** before other costs ($\Delta U > \Delta C$, eq. 5.69; break-even = 1,200/20 = 60 h saved). **Assumptions that could reverse it:** (1) the 75 h saving is an estimate. If the real reduction is below 60 h (for example, because reviewers still check every neural answer), the net becomes negative. (2) The \$20/h is only realised if the freed time is redeployed, and the \$1,200 excludes latency, monitoring, retraining and engineering time, which could exceed the \$300 margin.

Recommendation (≈ 125 words): investigate the data/label issue first, then test learned representations in Week 4. My audit found four false negatives. Two of them (q09_1, q09_3) come from passages labelled relevant by an index rule rather than by meaning, and only two (q06_1, q01_2) are genuine lexical paraphrases. A neural model compared against partly wrong labels would produce a misleading gain or loss. I would relabel the topics with a written guide and two reviewers, rerun the frozen ladder on validation, and then compare an embedding representation on a zero-overlap paraphrase slice. **Evidence that would change my mind:** if paraphrases still dominate the false negatives after relabelling, and embeddings recover them without new false positives, I would move straight to the Week 4 neural test.

Citations used: p. 420 (label definition, eq. 5.20); p. 464 (threshold cost); pp. 499–501 (paraphrase FN and ranking vs threshold errors); p. 502 (question hit rate); pp. 509–510 (Customer Support track); pp. 516–518 (acceptance review); p. 530 (ΔU vs ΔC).